TravelLink Backend — Progress Notes

Project Location

`F:\Projects\TravelLink\Server\Server\`

Tech Stack

- .NET 8, ASP.NET Core Web API
- Entity Framework Core + SQL Server (`TravelLinkDb`)
- JWT Authentication (`Microsoft.AspNetCore.Authentication.JwtBearer` v8.0.0)
- BCrypt.Net-Next (password hashing)

✅ Completed

Entities (`Models/Entities/`)

All 6 entities done:

- `User.cs` — includes `CreatedGroups` nav property
- `FriendRequest.cs`
- `Group.cs`
- `GroupMember.cs`
- `Expense.cs`
- `ExpenseSplit.cs`

AppDbContext (`Data/AppDbContext.cs`)

- All 6 DbSets registered
- Full Fluent API configured (dual FK Restrict on FriendRequest, Cascade on Group/Expense/ExpenseSplit)

Database

- `appsettings.json` — connection string + Jwt section (with `ExpiresInDays: 7`) all correct

- `InitialCreate` migration applied — `TravelLinkDb` live with all 6 tables

Program.cs

- DbContext registered
- JWT middleware registered
- `UseAuthentication()` + `UseAuthorization()` in correct order
- `IAuthService`, `IFriendService`, `IGroupService` all registered as Scoped

Auth Module ✅

- `DTOs/Auth/RegisterDto.cs`
- `DTOs/Auth/LoginDto.cs`
- `DTOs/Auth/AuthResponseDto.cs`
- `Services/IAuthService.cs`
- `Services/AuthService.cs` (Register, Login, GenerateToken)
- `Controllers/AuthController.cs` (POST /api/auth/register, POST /api/auth/login)
- **Tested & working ✅**

Friend System Module ✅

- `DTOs/Friends/SendFriendRequestDto.cs`
- `DTOs/Friends/FriendRequestDto.cs`
- `DTOs/Friends/FriendDto.cs`
- `Services/IFriendService.cs`
- `Services/FriendService.cs` (SendRequest, RespondToRequest, GetPendingRequests, GetFriends)
- `Controllers/FriendController.cs`
- **All 4 endpoints tested & working ✅**

Endpoint	Route
Send request	POST `/api/friend/send`
Accept/Reject	POST `/api/friend/respond/{id}?action=accept|reject`
Pending requests	GET `/api/friend/pending`

Friends list	GET `/api/friend/list`

Group Module ✅

- `DTOs/Groups/CreateGroupDto.cs`
- `DTOs/Groups/GroupDto.cs`
- `DTOs/Groups/GroupMemberDto.cs`
- `Services/IGroupService.cs`
- `Services/GroupService.cs` (CreateGroup, GetMyGroups, GetGroupById, AddMember, LeaveGroup, RemoveMember)
- `Controllers/GroupController.cs`
- **All 6 endpoints tested & working ✅**

Endpoint	Route
Create group	POST `/api/group/create`
My groups	GET `/api/group/my`
Get group by ID	GET `/api/group/{id}`
Add member (Admin only)	POST `/api/group/{id}/add-member`
Leave group	DELETE `/api/group/{id}/leave`
Remove member (Admin only)	DELETE `/api/group/{id}/remove-member`

🔧 Pending Modifications (before moving to Expense Module)

1. Auto-Delete Empty Groups

In `GroupService.LeaveGroupAsync`, after removing the member, check if the group has any members remaining. If `Members.Count == 0`, delete the group entity entirely.

```
// After _db.GroupMembers.Remove(member);
var remainingCount = await _db.GroupMembers.CountAsync(m => m.GroupId ==
groupId);
if (remainingCount == 0)
{
    var group = await _db.Groups.FindAsync(groupId);
    if (group != null) _db.Groups.Remove(group);
}
await _db.SaveChangesAsync();
```

> Same logic applies in `RemoveMemberAsync` — if Admin removes last member, group deletes.

---

2. Profile Picture Upload (User + Group)

Decision: Use Cloudinary (recommended for TravelLink)

Why Cloudinary over Azure Blob / Local Disk?

	Local Disk	Azure Blob	Cloudinary
Setup effort	Minimal	Medium	Low (SaaS)
Auto resize/optimize	No	No	Yes (auto WebP, thumbnails)
CDN delivery	No	Optional extra	Yes Built-in global CDN
Free tier	N/A	Limited	Yes 25GB/mo free
.NET SDK	N/A	Official	Official (CloudinaryDotNet)
Scalable	No	Yes	Yes

What We'll Need

- **NuGet:** `CloudinaryDotNet`
- **Config in `appsettings.json`:**

```
"Cloudinary": {
  "CloudName": "your-cloud-name",
  "ApiKey": "your-api-key",
  "ApiSecret": "your-api-secret"
}
```

- **DB Changes:** Add `ProfilePictureUrl` field to `User` entity (already has `CoverImageUrl` on Group)
- **New Migration** after schema change

Endpoints to Build

Endpoint	Route	Target
Upload/Update user avatar	POST `/api/user/avatar`	multipart/form-data
Upload/Update group cover	POST `/api/group/{id}/cover`	multipart/form-data
Get user profile	GET `/api/user/profile`	Returns user + avatar URL

Implementation Plan

1. Create Cloudinary account → get credentials
2. Install `CloudinaryDotNet` NuGet package
3. Add `ProfilePictureUrl` to `User.cs` entity + add migration
4. Register Cloudinary as a singleton in `Program.cs`
5. Build `IImageService` + `ImageService` (Upload, Delete old image)
6. Add avatar endpoint to a new `UserController`
7. Add cover image endpoint to existing `GroupController`

➡️ Next Steps (In Order)

1. **Apply pending modifications** (auto-delete empty group + profile picture feature)
2. **Expense Module** — Add expense, split logic (equal/custom), settle up, balances
3. **Real-time Location** — SignalR hub setup

4. **AI Itinerary** — Gemini API integration